Lab 7: Fuzzing for Input Validation Bugs

Software Testing and Analytics for Projects (SETAP)

Name: Harshith R Reddy
SRN: PES2UG24CS188
Section: D
Repository: github.com/HarShithRreddy/L6_L7_03_coffee_shop_order

---

Objective

Apply fuzz testing as a security and quality-assurance technique: use the **Hypothesis** property-based testing library to automatically uncover input-validation bugs in **processor.py**, then harden the code through defensive programming so it no longer crashes on unexpected input.

Bugs Found by Fuzzing

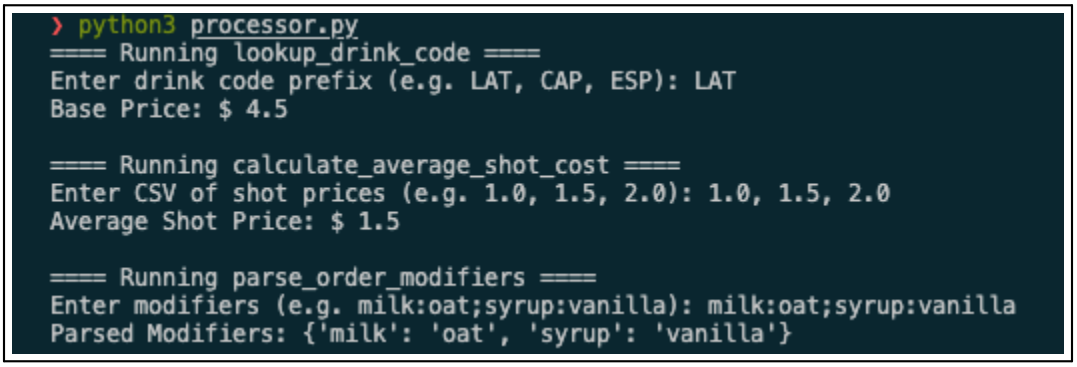
Function	Unhandled inputs discovered
`lookup_drink_code`	None → AttributeError on .upper(); unknown prefix → KeyError; the "MOc" key was unreachable (input is upper-cased to "MOC").
`calculate_average_shot_cost`	None → AttributeError on .split(); non-numeric / empty token → ValueError on float(); empty input → ZeroDivisionError.
`parse_order_modifiers`	None → AttributeError on .split(); an item with no ':' or an extra ':' → ValueError on tuple unpacking.

Fixes Applied (Defensive Programming)

Function	Hardening
`lookup_drink_code`	isinstance guard returns None for non-strings; dict .get() returns None for an unknown code instead of raising; "MOc" key corrected to "MOC".
`calculate_average_shot_cost`	non-string returns 0.0; empty and non-numeric tokens skipped via try/except; returns 0.0 when no valid prices (no division by zero).
`parse_order_modifiers`	non-string returns {}; items without ':' skipped; partition(':') handles extra colons safely.

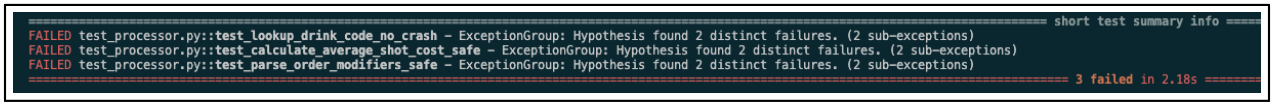
Screenshots

Step 2 — Run the buggy processor.py



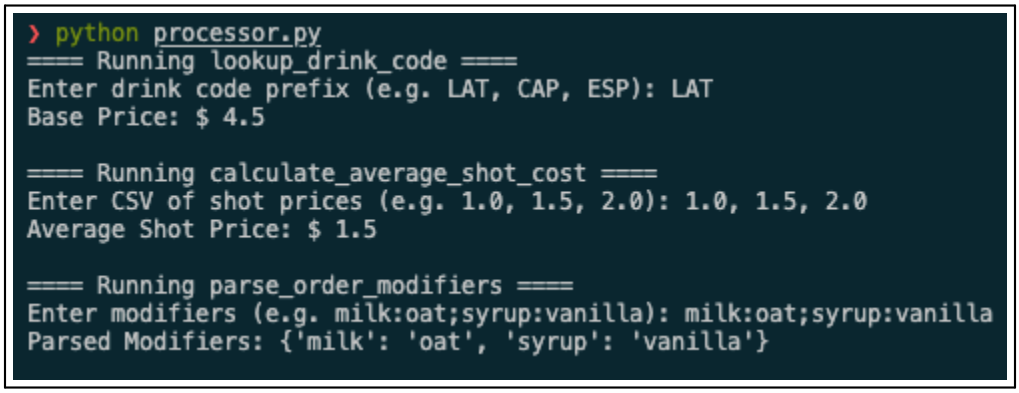
The program runs on valid input (LAT, a CSV of shot prices, and milk:oat;syrup:vanilla).

Step 4 — Fuzz tests against the buggy code (pytest)



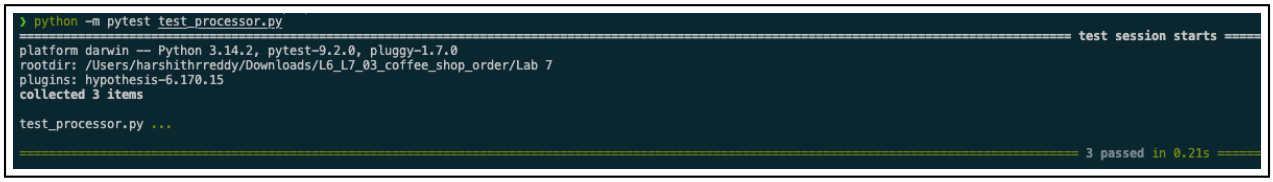
All 3 Hypothesis tests FAIL — the fuzzer found 2 distinct crashing inputs per function.

Step 5 — Run the fixed processor.py



After hardening, the program runs correctly on the same valid input.

Step 6 — Fuzz tests against the fixed code (pytest)



All 3 Hypothesis tests PASS — the functions no longer crash on any generated input.

Conclusion

Property-based fuzzing with Hypothesis automatically generated inputs that manual unit tests would likely miss — None values, unknown codes, non-numeric and empty tokens, and malformed modifier strings — exposing four distinct classes of unhandled exception. Adding input validation and defensive parsing to all three functions eliminated every crash, so the same fuzz suite that reported 3 failures now reports 3 passes, demonstrating improved robustness.